RTOS-Based Railway Block Control System

COMP 4900E - Real Time Operating Systems
Carleton University

Project Proposal

Group 5:
Peien Liu - peienliu@cmail.carleton.ca
Kina Zhan - kinazhan@cmail.carleton.ca
Samarth Arora - samartharora@cmail.carleton.ca
Jonathan Chan - jonathanchan@cmail.carleton.ca

Thesis Statement

A deterministic railway interlocking system built on QNX Neutrino that prevents deadlock and race conditions through centralized resource reservation and strictly prioritized message passing.

Team Roles and Responsibilities

Peien Liu:

- Design and implement interlocking logic for a deadlock prevention engine to prevent deadlocks and unsafe train movements across various deadlock types.
- Test and validate deadlock engine and safety enforcement.

Kina Zhan:

- Develop a train physics engine to simulate speed, braking, and route compliance.
- Design a predictive component for the deadlock prevention engine to predict deadlock states
- Test and validate real-time task deadlines and train behavior.

Samarth Arora:

- Develop a routing engine to route/reroute multiple trains within a deadlock region.
- Simulate track layout, block occupancy, switches, and sensor events for testing.
- Implement logging and operator interface for monitoring trains, signals, and events.

Jonathan Chan:

- Configure real-time scheduling, process priorities, and IPC on QNX to ensure deterministic behavior.
- Create communication engine to handle client (train) - server (RTOS) information exchange as well as regionized information advertisement (if handling neighboring deadlock regions).

Domain Description

Railway control systems coordinate trains over shared track by controlling signals, switches, and track access permissions to keep traffic safe and efficient. In practice, this means continuously

tracking which track blocks are occupied, deciding which routes can be reserved, and ensuring that only non-conflicting movements are authorized at any moment. On single-track lines, multiple trains may request the same block at the same time, and without strict coordination this can lead to serious outcomes such as head-on conflicts of two trains or a permanent stand-off where neither train can proceed. To prevent this, we want to design an interlocking railway control systems system that serves as a centralized safety authority that enforces rules for block occupancy, route locking, switch alignment, and signal aspects so that conflicting routes cannot be granted to trains simultaneously. Because these decisions are time-sensitive and safety-critical, modern implementations often rely on real-time operating systems to guarantee deterministic communication, priority scheduling, and timely fault handling.

Purpose of the Application

This application solves the risk of head-on collisions and permanent deadlocks in railway networks by implementing an RTOS-based interlocking system. We model our simulation after the Ottawa O-Train network, focusing on the transition between the double-tracked Line 1 and the single-tracked Line 2. This topology creates bottlenecks where trains compete for limited track segments. By utilizing QNX Neutrino, the system enforces centralized resource reservation and prioritized messaging to ensure that conflicting routes are never granted. The application proves that deterministic scheduling can guarantee safety and maintain liveness under strict real-time deadlines.

Development Technology

Components: QNX Neutrino RTOS, Raspberry Pi (hosting QNX), Makefile, Git/GitHub, Momentics IDE, C, Python, Docker

The implementation of the RTOS-based railway interlocking system relies on a carefully chosen set of tools and infrastructure that balance real-time correctness, reproducibility, and developer productivity. At the core of the system is QNX Neutrino RTOS which provides deterministic, priority-based preemptive scheduling and synchronous message passing. These features are essential for modeling safety-critical interlocking logic, where missed deadlines or race conditions could translate into deadlocks on the tracks. QNX’s microkernel architecture allows

the safety logic, hardware abstraction, and diagnostics to run as isolated processes, improving fault containment and enabling the system to fail safely if a component misbehaves.

The RTOS is hosted on a Raspberry Pi which serves as a realistic embedded deployment platform rather than a purely simulated environment. Running QNX on actual hardware allows the project to validate timing guarantees, interrupt handling, and scheduling behavior under constrained resources, closely mirroring real railway controllers. This hardware choice also keeps the system accessible and cost-effective for development while still supporting meaningful real-time experimentation.

The system is implemented primarily in C, using native QNX libraries for interprocess communication, timers, and synchronization primitives. C is chosen for its low runtime overhead, fine-grained memory control, and predictable execution characteristics. Code predictability and reliability is a must when handling deadlock prevention. Supporting infrastructure code, simulation orchestration, and data analysis are written in Python. Although Python does not offer the same guarantees as C, its logic will stay outside of the core deadlock prevention logic and act as support for diagnostics, simulation, and other server side applications.

Build and deployment are managed using a Makefile, which automates compilation, linking, and configuration of our C development. This ensures consistent compiler flags, reproducible builds, and clear dependency management for easier traceability and maintainability standards. For version control and collaboration, the project uses Git with remote hosting on GitHub. This setup provides traceability of changes, supports parallel development among team members, and enables code reviews, following professional standards. GitHub also provides a source of truth to prevent differentiating code by denying merge conflicts.

Development and debugging are performed using the QNX Momentics IDE, which integrates cross-compilation, remote debugging, and runtime monitoring for QNX targets. Momentics allows developers to inspect thread priorities, message queues, and scheduling behavior in real time, making it easier to verify that high-priority safety tasks are never starved by lower-priority activities. Finally, Docker is used to containerize services such as Python-based servers or simulation tools. Containerization ensures a consistent execution environment across different development machines and simplifies setup for demonstrations. Together, these tools form a

cohesive infrastructure that supports deterministic execution, rigorous testing, and collaborative development.

Study Method

The proposed study will employ a simulation-based approach within the QNX Neutrino Real-Time Operating System to validate the efficacy of centralized block reservation in preventing railway deadlocks. To benchmark the system's logic, we will compare our real-time deadlock prevention mechanisms against the theoretical models presented in "The Tick Formulation for deadlock detection and avoidance in railways traffic control". While their research utilizes a model that detects states that are bound to deadlock—situations where trains typically move into positions from which they cannot advance —our project will adapt these concepts for dynamic, real-time control. We will specifically use a "culprit set", a set of the specific trains responsible for a deadlock, to design stress tests that would evaluate whether or not the RTOS scheduler can identify and halt these trains before a deadlock becomes irreversible.

The simulation will feature configurable train models defined by key attributes such as speed, weight, and length, which are essential for determining track occupancy. As noted in the reference research, deadlocks in single-track networks are frequently caused by trains that are too long to find a station between them in a way that they can be routed past one another. By varying the length and speed parameters in our train models, we can simulate scenarios where a standard siding is insufficient for a specific train, thereby testing the system's ability to dynamically reject block reservations that would lead to a blocked main line. This allows for a flexible range of test cases without requiring a rigid physics engine, focusing instead on the logic of resource allocation.

To evaluate track types and deadlock scenarios, the study will model a network inspired by the Ottawa O-Train, specifically focusing on the operational transition between the double-tracked Line 1 and the single-tracked Line 2. This setup will attempt to mirror single-line networks, where deadlocks are most common due to the shared use of tracks for both directions. The interchange between the lines will serve as the boundary nodes, representing the points where trains enter and exit the controlled single-track territory. This topology provides a natural

environment to test and help detect conflicts (trains competing for the same single track) and determine if a safe place exists for waiting trains.

Testing procedures will distinguish between single deadlock scenarios and more complex multiple deadlock chains. Initial validation will focus on simple two-train conflicts, which are the most frequent form of deadlock. Subsequent phases will attempt to induce multiple deadlock scenarios involving three or more trains. Research indicates that in practical single-line operations, deadlocks caused by more than three trains are negligible but possible. By simulating circular wait conditions, we can verify if the RTOS-based controller can successfully maintain flow or if it requires manual intervention, providing a comprehensive analysis of the system's safety limits.

Timeline/Delivery Deadlines

Phase 1: Initial Research and Requirement Analysis

- Confirm the track scenario (single track + passing siding) and assumptions.
- Define safety and liveness requirements (no collisions, no permanent deadlock).
- Finalize evaluation criteria and success metrics.

Phase 2: System Design and Implementation

- Design the overall system architecture and interfaces.
- Implement the interlocking controller and basic train simulation workflow.
- Prepare a demo scenario that matches the chosen topology.

Phase 3: Testing, Verification, and Performance Analysis

- Run test scenarios (conflicting requests, opposite directions, high contention).
- Verify safety invariants and demonstrate deadlock prevention.
- Measure timing/latency and scheduling behavior under load.

Phase 4: Documentation and Demo

- Write report: model, policy, architecture, test results, timing graphs
- Prepare demo script + slides
-

Github Repo

https://github.com/PayanLPE/RTOS-Railway-Block-Control-System

References

[1] V. Dal Sasso, L. Lamorgese, C. Mannino, A. Onofri, and P. Ventura, "The Tick Formulation for deadlock detection and avoidance in railways traffic control," *J. Rail Transp. Plan. Manage.*, vol. 17, p. 100239, Mar. 2021, doi: 10.1016/j.jrtpm.2021.100239.

[2] L. Zhao, C. Wen, and C. P. L. Lau, "A review of railway signalling system development and its future," *Transp. Res. Part C: Emerg. Technol.*, vol. 51, pp. 1–17, Feb. 2015, doi: 10.1016/j.trc.2014.10.007.

[3] I. G. S. G. P. Putra, J. G. de Bruijn, J. G. van de Pol, and M. Stolikj, "Formal modeling and verification of railway interlocking systems: A survey," *Transp. Res. Interdiscip. Perspect.*, vol. 20, p. 100826, July 2023, doi: 10.1016/j.trip.2023.100826.

[4] City of Ottawa, "PART 3 DESIGN AND CONSTRUCTION REQUIREMENTS – SYSTEMS," in *Project Agreement – Schedule 15-2, Trillium Line Extension, Ottawa Stage 2 LRT Project*, Execution Version, 2019. [Online]. Available: https://documents.ottawa.ca/sites/documents/files/TLPA-Schedule%2015-2%20Part%203%20-%20Systems%20%28Redacted%29-AODA.pdf

[5] Transport Canada, "Railway Signal and Traffic Control Systems Standards," TC E-7.1, Aug. 26, 1996. [Online]. Available: https://tc.canada.ca/en/rail-transportation/standards/railway-signal-traffic-control-systems-standards. [Accessed: Feb. 14, 2026].

[6] "Automatic Train Control," *The Railway Technical Website*. [Online]. Available: http://www.railway-technical.com/signalling/automatic-train-control.html. [Accessed: Feb. 14, 2026].

[7] "Train operation 101: How trains are controlled," *Trains Magazine*, ABCs of Railroading. [Online]. Available: https://www.trains.com/trn/train-basics/abcs-of-railroading/train-operation-101-how-trains-are-controlled/. [Accessed: Feb. 14, 2026].

[8] P. H. Wallen, "Raspberry Pi Model Railway Automation," *phwallen.github.io*. [Online]. Available: https://phwallen.github.io/smrc/. [Accessed: Feb. 14, 2026].